



Entornos de Desarrollo

*UT3- Diseño y realización de
pruebas*



Pruebas de software

Consisten en el **proceso de evaluación** de un programa o sistema informático para identificar si su **funcionamiento es el esperado**, si **cumple con los requisitos** del usuario y si se **comporta correctamente** en distintas situaciones

Es importante destacar que el objetivo de las pruebas **NO** es asegurar **la ausencia de errores** en un software (lo único demostrable es que existan defectos en él), sino **diseñar pruebas** que **sistemáticamente muestren diferentes clases de errores**, haciéndolo con la menor cantidad de tiempo y esfuerzo



Pruebas de software

Para aumentar la probabilidad de encontrar errores se deberían hacer pruebas tanto por el equipo desarrollador, como por un equipo independiente, que no esté tan familiarizado con la estructura. Una buena prueba debe:

- Tener una alta probabilidad de encontrar un fallo
- Centrarse en dos objetivos:
 - Probar si no hace lo que debe hacer
 - Probar si hace lo que no debe hacer
- No debe ser redundante, propósitos definidos (pérdida de tiempo y recursos)
- Descubrir una clase/grupo entero de errores
- No debe ser ni muy sencilla ni muy compleja pero con un único objetivo



Tareas en las Pruebas de software

Las tareas a realizar para probar un software serían:

1. **Diseño de las pruebas:** identificación de la técnica o técnicas de pruebas que se utilizarán para probar el software
2. **Generación de los casos de prueba:** Los casos de prueba representan los datos que se utilizarán como entrada para ejecutar el software a probar.
Más concretamente los casos de prueba determinan un **conjunto de entradas, condiciones de ejecución y resultados esperados para un objetivo particular**. La generación de cada caso de prueba debe ir acompañada del resultado que ha de producir



Tareas en las Pruebas de software

3. **Definición de los procedimientos de la prueba:** especificación de cómo se va a llevar a cabo el proceso, quién lo va a realizar, cuándo, condiciones...
4. **Ejecución de la prueba:** aplicando los casos de prueba generados previamente e identificando los posibles fallos producidos al comparar los resultados esperados con los resultados obtenidos
5. **Realización de un informe de la prueba:** con el resultado de la ejecución de las pruebas, qué casos de prueba pasaron satisfactoriamente, cuáles no, y qué fallos se detectaron para su inmediata **depuración**



Pruebas de software

Antes de continuar es importante tomar conciencia de que, durante el proceso de desarrollo de software, se pueden producir dos tipos de errores claramente identificables:

Errores de codificación: el mismo entorno nos proporciona información de dónde se produce y cómo solucionarlo. El programa no puede compilarse hasta que no se corrija estos errores:

- Uso incorrecto de una sintaxis o función en el código
- Olvidar cerrar una etiqueta o llave en el código
- Declarar una variable con un tipo de dato incorrecto
- Error al asignar un valor a una variable
- Error al pasar parámetros a una función
- Escribir mal un nombre de variable o función



Pruebas de software

Errores lógicos o bugs: no evitan que el programa se pueda compilar con éxito, sin embargo, pueden provocar que **el programa devuelva resultados erróneos**, que no sean los esperados o pueden provocar que el programa termine antes de tiempo o que se quede en bucle infinito. Por ejemplo:

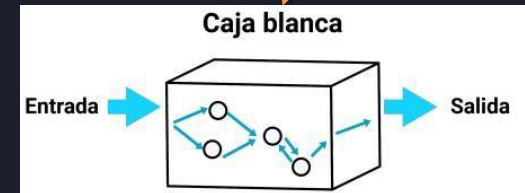
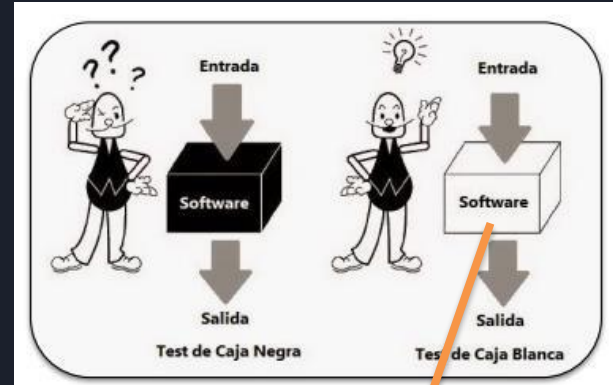
- Una condición lógica en el código que no se cumple correctamente
- Un bucle que no se ejecuta el número correcto de veces
- Un algoritmo que no devuelve el resultado esperado
- Una función que devuelve un resultado incorrecto
- Un objeto o variable que no se inicializa correctamente
- Una excepción que no se maneja correctamente

Técnicas - Pruebas de software

Hay diferentes **Técnicas** para realizar estas pruebas:

Caja negra: (funcionales) el programa o aplicación es probada observando los resultados a partir de las entradas, sin tener en cuenta el funcionamiento interno. Solo se debe conocer la funcionalidad

Caja Blanca: (estructurales) se hace un examen del código y la estructura del programa que se va a probar por lo que se debe conocer la lógica





Pruebas de Caja Blanca - criterios

Para llevar a cabo estas pruebas se han definido distintos **criterios** que permiten decidir qué sentencias o caminos se deben examinar con los casos de prueba. En cada caso hay que **escribir casos de prueba suficientes para lograr el objetivo**:

- **Cobertura de Sentencias**: cada sentencia en el programa se ejecute al menos una vez
- **Cobertura de Decisión**: se trata de crear los suficientes casos de prueba para que cada opción resultado de una comprobación lógica del programa, se evalúe al menos una vez a cierto y otra a falso. En la decisión MIENTRAS (A and B), habrá casos de prueba donde (A and B) sea verdadero y donde (A and B) sea falso.



Pruebas de Caja Blanca - criterios

- **Cobertura de Condiciones:** se trata de crear los suficientes casos de prueba para que cada elemento de una condición, se evalúe al menos una vez a falso y otra a verdadero. En la decisión MIENTRAS (A and B), habrá casos de prueba donde A sea falso, A sea verdadero, B sea falso y B sea verdadero.
- **Cobertura Decisión/Condición:** consiste en cumplir simultáneamente las dos anteriores.
- **Cobertura de Caminos:** se ejecuten todos los caminos de un programa. Entendiendo camino como una secuencia de sentencias encadenadas desde la entrada del programa hasta su salida

Pruebas de Caja Negra - criterios

Al igual que con las técnicas de Caja Blanca, para confeccionar los casos de prueba de Caja Negra existen distintos criterios:

- **Particiones de Equivalencia:** considerar el menor número posible de casos de pruebas, por lo que, cada caso de prueba tiene que abarcar el mayor número posible de entradas diferentes



- **Análisis de Valores Límite:** se van a elegir como valores de entrada, aquellos que se encuentra en el límite de las clases de equivalencia



Pruebas de Caja Negra - criterios

- **Pruebas aleatorias:** consiste en generar entradas aleatorias para la aplicación que hay que probar. Se suelen utilizar generadores de prueba, que son capaces de crear un volumen de casos de prueba al azar, con los que será alimentada la aplicación.
- **Conjetura de errores:** trata de generar casos de prueba que la experiencia ha demostrado generan típicamente errores. En valores numéricos, un buen ejemplo es comprobar si funciona correctamente con el valor 0, ya que si es utilizado como denominador en alguna división podría generar un error en nuestro programa.

Pruebas de software - Estrategia

La estrategia que se ha de seguir a la hora de evaluar dinámicamente un sistema software debe permitir **comenzar por los componentes más simples** y más pequeños e **ir avanzando progresivamente hasta probar todo en su conjunto**. De tal manera que lo conveniente sería realizar las pruebas en el siguiente orden:

- Pruebas **unitarias**: son pruebas enfocadas en verificar el comportamiento de cada componente individual del sistema de manera aislada
- Pruebas de **integración**: se realizan para comprobar el correcto funcionamiento de los componentes cuando se integran en el sistema completo



Pruebas de software - Estrategia

- Pruebas de **sistema**: se centran en evaluar el comportamiento del sistema completo y su interacción con otros sistemas y entornos.
- Pruebas de **aceptación/validación**: son pruebas realizadas por los usuarios o clientes finales para verificar si el sistema cumple con sus requisitos, necesidades y expectativas
- Pruebas de **regresión**: Buscan detectar fallos introducidos durante la modificación de un sistema o componente de un sistema
- Pruebas de **rendimiento**: evalúan el desempeño del sistema bajo diferentes cargas de trabajo
- Pruebas de **seguridad**: se enfocan en identificar vulnerabilidades y riesgos de seguridad del sistema



Pruebas de software - Herramientas

Todos los entornos de desarrollo, incluyen una serie de **herramientas de depuración**, que nos permiten verificar el código generado, ayudándonos a realizar pruebas tanto estructurales como funcionales, o de algún otro tipo:

Puntos de interrupción/ruptura: (breakpoint) permiten pausar la ejecución del programa en un punto específico para examinar el estado actual de las variables y objetos

Paso a paso: permiten ejecutar el programa paso a paso, revisando la ejecución de cada línea de código y el valor de las variables en ese momento



Pruebas de software - Herramientas

Monitoreo de variables: (watch) permiten examinar el valor de una variable o conjunto de variables mientras el programa se está ejecutando

Evaluación de expresiones: permite evaluar una expresión mientras el programa se está ejecutando, para ver su valor actual

Registro de eventos: permite registrar la ejecución de eventos específicos en el programa, como llamadas a funciones o excepciones, para examinar su comportamiento